

ESTRUCTURAS DE CONTROL ITERATIVAS

El siguiente cuadernillo, se presenta como guía rápida de la sintaxis de las estructuras de control iterativas y sus aplicaciones en el lenguaje Python. Los temas expresados, son ampliados en los videos tutoriales y en las clases presenciales.

Una estructura de control, se define de la siguiente forma:

inicio de la estructura de control: sentencias

El inicio de la estructura de control está dado por una expresión lógica, en la que las variables de la misma, deben ser definidas previamente, antes de ingresar al bloque de expresiones.

Recordar que, **la indentación es obligatoria**, de 4 (cuatro) espacios, ya que, de ella, dependerá la funcionalidad de la estructura.

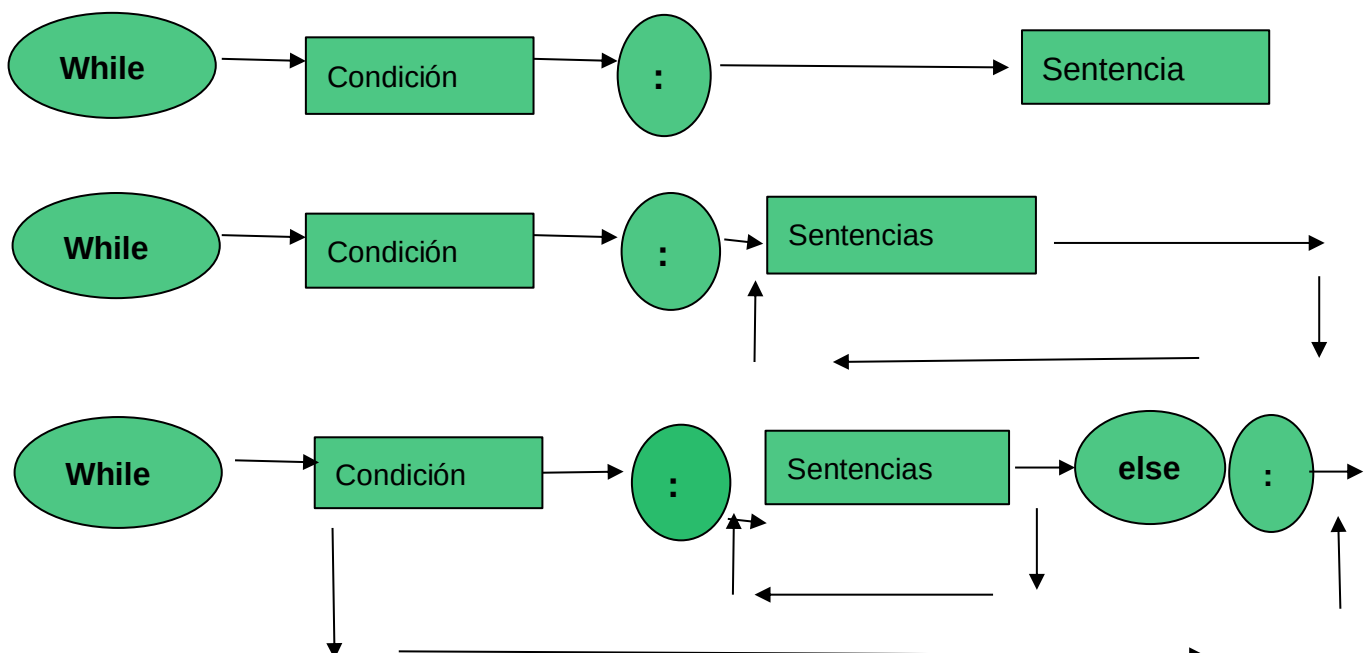
Estos espacios indicarán que las instrucciones indentadas, forman parte de una misma estructura de control.

Dos son las estructuras de control de repetición o iterativas, en Python:

- Iterativa Pre-Condicional: **while**.../ Mientras...
- Iterativa Condicional: **for**.../Para...

Estructura Iterativa Pre-Condicional: **while** ... / Mientras...

a) Sintaxis:



Codificación:**a) definir variables de expresión_lógica**

```
while expresión_lógica:  
    sentencia1  
    sentencia2
```

En la expresión_lógica, sus variables deben estar definidas, previo al ingreso de la estructura.

El bloque de sentencias indentadas, se ejecutará mientras la expresión_lógica sea **Verdadera**. Cuando ésta, sea actualizada y sea Falsa, finaliza el bucle iterativo.

b) while expresión_lógica:

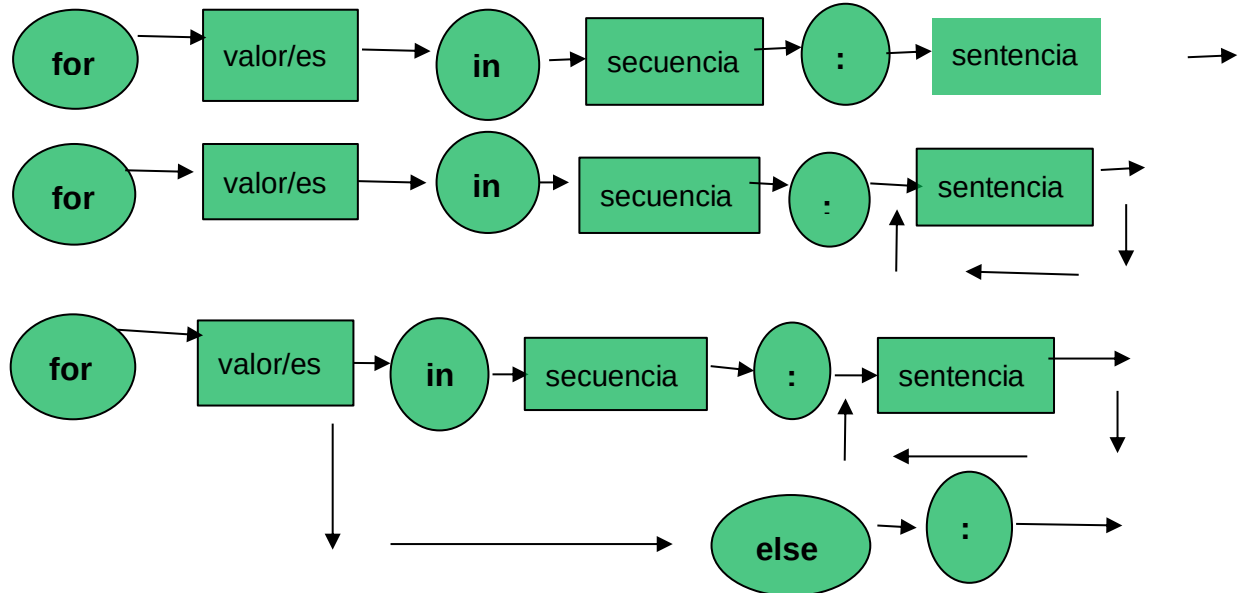
```
    sentencia1  
    sentencia2  
    ...  
else:  
    sentencia1  
    sentencia2  
    ...  
sentencia
```

Al funcionamiento descrito del ciclo del while, se le agrega que al finalizar se ejecutará el **else:** (optativo) que permite ejecutar un bloque de sentencias determinado, de lo contrario continuará con la secuencia del algoritmo.

*No serán usadas en el cursado, las sentencias **break** y **continue**, pues no compatibilizan con el paradigma estructurado, al cambiar la secuencia lógica sin considerar la condición de fin de las estructuras iterativas.*

Estructura Iterativa Condicional: **for...(Para)**

Sintáxis



a) **for** valor **in** secuencia:
 sentencia1
 sentencia2
 ...

El ciclo del **for**, itera sobre una secuencia (listas, tuplas, cadenas, o cualquier objeto iterable), la cantidad determinada de veces, que el valor se encuentra en ella, hasta finalizar la secuencia.

b) **for** valor **in** secuencia:
 sentencia1
 sentencia2
 ...
 else:
 sentencia1
 sentencia2
 ...
 sentencia

Al funcionamiento descrito del ciclo del **for**, se le agrega que, al finalizar el bucle, se ejecutarán las sentencias declaradas después de **else:** antes de continuar con la secuencia lógica del programa.

Para la secuencia del **for**, es muy utilizada la función **range ()**, que está integrada al lenguaje, facilitando la funcionalidad de esta sentencia.

Ejemplo:

```
>>> for k in range (5):  
>>>     print(k)  
0  
1  
2  
3  
4
```

Función `range ()`

La función retorna como resultado números enteros, en secuencia.

Como parámetros por valor tiene un **Valor_inicial**, **Valor_final** y **Valor_de_incremento**.

Casos:

- Si se define un parámetro `n` correspondiente a un número entero, retorna como resultado una secuencia inmutable de números enteros., empezando en 0 y finalizando con `n-1`.

Podemos observarlo, con la función `list ()` del lenguaje:

```
>>> list (range (6))  
0,1,2,3,4,5  
>>> list (range (0))  
[ ]  
>>> list (range (0,1))  
[ ]
```

- Si se especifican 2 parámetros, el primero identifica el inicio de la sucesión. Observar que el rango incluye el `Valor_inicial` y no incluye el `Valor_final`.

Ejemplo:

```
>>> list (range (1, 10))  
1,2,3,4,5,6,7,8,9
```

- Si se especifican 3 parámetros, el tercero identifica el intervalo entre 2 números de la sucesión.

Ejemplo:

```
>>> list (range (1,10, 3))  
1,4,7
```